



Crochetify

CODE YOUR PATTERN;

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

22 / 06 / 2025

INTEGRANTES:

VALENTIN GARFI - 64486

TOMÁS BORDA - 64517

NICOLÁS ARANCIBIA - 64481

PROFESORES:

ARIAS ROIG, ANA MARÍA

GOLMAR, MARIO AGUSTÍN

ADER, MANUEL

MORRONI, FRANCO



Tabla de contenidos

1. Introducción	3
2. Modelo Computacional	3
2.1. Dominio	3
2.2 Lenguaje	5
3. Implementación	7
3.1. Frontend	7
3.2. Backend	8
3.3. Adicionales	8
3.4. Dificultades Encontradas	9
3.5. Modificaciones respecto al diseño inicial	10
4. Futuras extensiones	11
5. Conclusiones	12
6. Referencias	12
7. Bibliografía	12

1. Introducción

Crochetify es un lenguaje que permite la construcción de patrones de *filet crochet*. Mediante éste es posible generar imágenes de patrones de crochet descritos de forma imperativa, mediante el uso de diversas funcionalidades provistas por el lenguaje como definición de patrones, parametrización de los mismos, definición de variables y funciones para repetir y espejar patrones.

La salida del mismo es una imagen con el patrón deseado, generada con *python* haciendo uso de la librería *pillow* para la construcción de la misma.

En el presente documento, se detalla el funcionamiento y desarrollo del mismo, las dificultades encontradas en el proceso y las posibles futuras implementaciones.

2. Modelo Computacional

2.1. Dominio

El lenguaje permite la creación de patrones de *filet crochet*. Esta técnica dentro del *crochet* abarca los siguientes puntos: **cadena**, **crochet simple o punto bajo** y **crochet doble o punto alto**. Sus representaciones, son visibles en la tabla que se muestra a continuación:

	Cadena (CH)	Punto Bajo (SC)	Punto Alto (DC)
Símbolo			

Tabla 1: Símbolo usado para representar cada uno de los puntos descritos.

Asimismo, el *filet crochet* tiene la característica de que los patrones se realizan con filas rectas y formando una figura que se asemeja a una grilla (Ver figura 1).



Figura 1: Ejemplo de un patrón de filet crochet, utilizando cadenas y puntos altos.

En contraparte, existen patrones de crochet cuya forma es más bien circular o indefinida en ciertos casos, lo que complejiza aún más la construcción del mismo de forma imperativa (Ver figura 2).

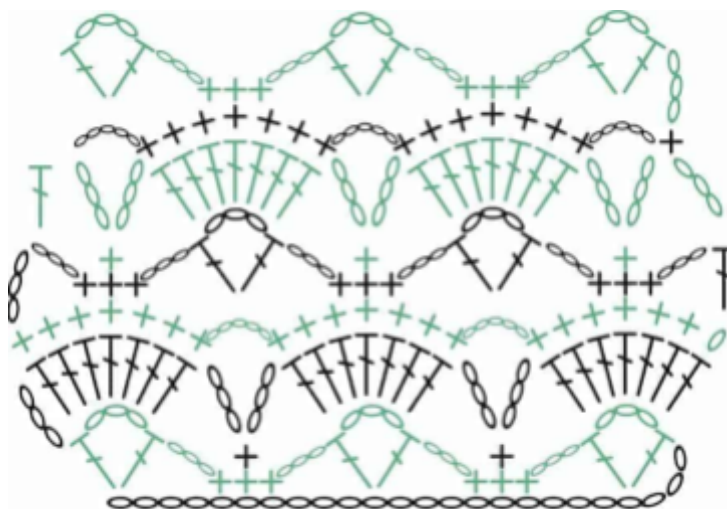


Figura 2: Ejemplo de un patrón de crochet más complejo.
Fuente: Joy of Motion Crochet, *Reading Crochet Charts*. Disponible en:
<https://joyofmotioncrochet.com/reading-crochet-charts/>

Crochetify se limita únicamente a la construcción de patrones como los observables en la Figura 1. Éstos tienen una forma particular de ser leídos: siempre se comienza desde la fila que esté lo más abajo posible, leyendo de izquierda a derecha los puntos. En la posición de inicio de lectura debería haber un símbolo de un punto que indique el inicio de la lectura. Cuando doy la vuelta, invierto el sentido de lectura. A continuación, se presenta una figura detallando el sentido de lectura para un patrón de filet crochet:

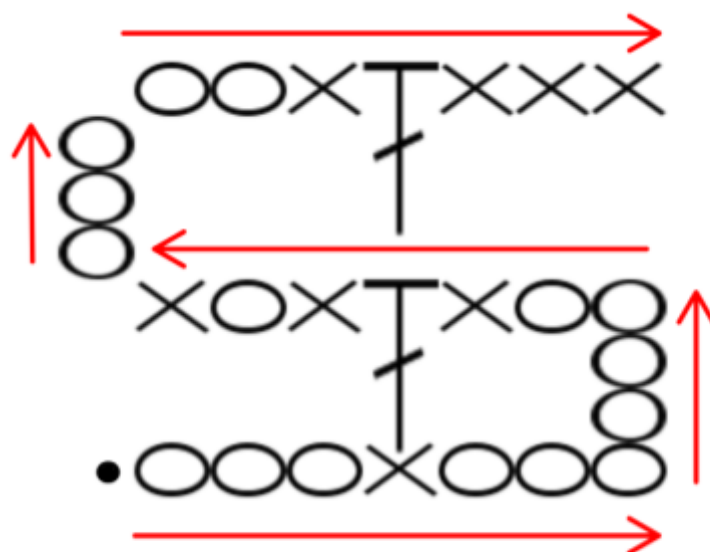


Figura 3: Patrón de crochet que tiene indicado el sentido de la lectura, comenzando desde el punto y siguiendo el sentido de las flechas en rojo

Algo a tener en cuenta es que no todos los puntos pueden colocarse en cualquier momento: Los puntos altos (DC), por ejemplo, requieren haber subido con 3 puntos, mientras que los bajos (SC) con 1. Así también, las vueltas pueden darse únicamente con puntos de tipo cadena.

Por último, como se puede observar en el último patrón de ejemplo (Ver Figura 3), el último punto de una fila determina de qué manera se dibujará la vuelta: Si es por fuera o si se debe realizar por encima del último punto. Esto implica que un patrón de crochet filet estará bien definido si la longitud de cada fila coincide, en cantidad de puntos, con la cantidad de puntos sobre la hilera base del crochet. Se debe tener en cuenta que los puntos que se consideran como parte de la base del patrón son únicamente aquellos posicionados en la hilera horizontal, mas no aquellos que se dibujen por fuera y que formen parte de la vuelta.

2.2 Lenguaje

La construcción del patrón de crochet, como se mencionó anteriormente, se realiza de manera imperativa, indicando fila por fila qué puntos dibujar y en qué momento dibujar una vuelta. El sentido de dibujo es el mismo que el de lectura, pues el lenguaje asume que quién construye el patrón conoce lo descrito en la sección anterior acerca de cómo leer un patrón de crochet y sabe que, por ejemplo, luego de dar una vuelta, el sentido de dibujo se invierte. Así también, los puntos son descritos con: **CH** para cadena, **SC** para punto bajo y **DC** para punto alto.

Crochetify ofrece las siguientes funcionalidades:

- I. Descripción de una fila o vuelta con símbolos CH, SC y DC.
- II. Indicación de una vuelta mediante la directiva TURN.
- III. Designación de un color para una fila al inicio de esta, ya sea vuelta o fila horizontal. En caso de no indicarlo, la misma será de color negro.
- IV. Definición de patrones complejos con varias filas y parametrizables, pudiendo recibir puntos, colores y otros patrones.
- V. Utilización de una estructura de control REPEAT para repetir un patrón un número arbitrario de veces.
- VI. Utilización de una estructura de control MIRROR para espejar patrones un número arbitrario de veces.
- VII. Directivas BEGIN CROCHET y END CROCHET que indican el patrón a dibujar.
- VIII. Declaración de variable de tipo *Stitch* o *Color* previo a BEGIN CROCHET.

Un *patrón de crochet bien formado* es aquel que no posee dos filas de vuelta consecutivas, así como tampoco filas cuya cantidad de puntos no llega a la longitud esperada para poder dar la vuelta. Asimismo, un patrón bien formado tampoco debe tener puntos altos o bajos en una fila si no se dio una vuelta con la cantidad de puntos necesaria. Se pueden ver a continuación un patrón bien formado (Ver Figura 4) y otro mal formado (Ver Figura 5):

```
BEGIN CROCHET
CH CH CH;
TURN CH CH CH; // Se da la vuelta con 3 puntos cadena
CH DC;
/*
    Se puede colocar DC ya que se dió la vuelta con 3 puntos cadena
    Además, solo se necesitan 2 puntos ya que al terminar la fila
    anterior con CH la vuelta se realiza por encima de la última cadena
*/
END CROCHET
```

Figura 4: Se puede observar el código de un patrón bien formado con comentarios al respecto.

```
BEGIN CROCHET
CH CH CH;
TURN CH CH; // Se da una vuelta con 2 puntos cadena
CH DC;
/*
    No se puede colocar DC ya que se dio una vuelta con 2 puntos
    cadena y se necesitan 3. Además, se necesitan 2 puntos para completar
    la fila y poder dar la vuelta, y solo se tiene 1, por lo que el
    patrón está mal formado
*/
END CROCHET
```

Figura 5: Código de un patrón mal formado con los motivos comentados en él.

El código que describe el dibujo final del programa, que es el que queda entre las directivas `BEGIN CROCHET` y `END CROCHET`, exige que éste sea un patrón bien formado.

3. Implementación

3.1. Frontend

El frontend comienza con el análisis de la entrada utilizando Flex para la división en Tokens. Se manejan 5 contextos diferentes: **INITIAL**, **MUTLILINE_COMMENT**, **INLINE_COMMENT**, **PATTERN_DEFINITION** y **CROCHET**. Los contextos de comentarios se utilizan para permitir comentar el código, ignorando todo el contenido entre ellos. Por otro lado, **PATTERN_DEFINITION** y **CROCHET** son los contextos que corresponden al cuerpo de un patrón definido y al del crochet principal respectivamente.

A continuación, en Bison se establece la gramática con las respectivas producciones. Por ejemplo, se establece que no se pueden hacer declaraciones después de un `BEGIN CROCHET`, sino que todas deben estar previamente definidas, ya sean colores, puntos o patrones. Así también, se maneja una tabla de símbolos por scope, donde se inicia un nuevo scope cada vez que se ingresa a la definición de un patrón. Para la tabla de símbolos se utilizó una librería externa llamada *khash*, de modo tal de poder implementar una estructura de *mapa* donde las entradas del mismo son pares (clave, valor), pudiendo ingresar y acceder a las mismas de forma eficiente. Durante la etapa de análisis sintáctico se evalúan, consecuentemente, los



identificadores hallados para validar no solo que estos existan, si no también que en ciertos casos sean del tipo esperado.

El programa final se divide en *declaraciones* y *cuerpo principal*, teniendo en éste último todas las instrucciones descritas entre BEGIN CROCHET y END CROCHET mientras que en la primera se encuentran todos los identificadores definidos con sus respectivos valores. Esto, junto al acceso a la tabla de símbolos, es lo que se recibe en el backend para realizar el posterior análisis semántico.

3.2. Backend

El objetivo final del análisis semántico es poder proveer una estructura *CrochetResult* que indique si el análisis fue exitoso (en caso de que el cuerpo principal del programa, luego de todas las expansiones necesarias, esté bien formado) y contenga una estructura *RowNodeList* con todas las filas del crochet y los puntos en cada una de ellas, de forma tal que se pueda generar el código fácilmente leyendo fila por fila y generando punto por punto.

Para ello se cuenta con diferentes funciones que se encargan de computar cada una de las filas a partir de leer las estructuras necesarias, recorriendo nuevamente el árbol en caso de ser necesario o buscando las declaraciones pertinentes. Cada una de estas funciones aporta lo que le corresponde a la estructura final que terminará siendo leída para generar el código. Asimismo, es esta la etapa en la que se realizan chequeos a medida que se expande el crochet final, verificando por ejemplo que la cantidad de puntos de cada fila sea la adecuada dependiendo de cómo se haya subido y de cuántos puntos tenía la fila de debajo, así como también que no haya dos instrucciones TURN consecutivas o, por el contrario, dos filas sin TURN en el medio.

Para la salida final del programa y la generación de la imagen se utiliza Python, en particular una librería llamada *Pillow*. El código para la generación de la imagen sigue el modo imperativo en el que se describe el crochet, dibujando los puntos y moviendo el cursor en la siguiente dirección.

De esta manera, cualquier aficionado del crochet filet puede utilizar Crochetify para confeccionar sus patrones favoritos. La salida generada está lista para ser impresa y tomar como referencia para tejer las mejores piezas de lana.

3.3. Adicionales

Como se mencionó anteriormente, la salida final del procesamiento de un código con el script *start.sh* es una imagen. Modificando el script y comentando la sección donde se borra el

archivo de salida en *python* generado (Ver Figura 6), podrá visualizarse el verdadero código de salida que posteriormente es ejecutado para la generación del patrón de crochet final.

```
sudo rm awesome_crochet.png # temp_generated_script.py
# Agrega '#' antes de temp_generated_script.py
```

Figura 6: Muestra la línea que debe de comentarse en *start.sh* en caso de querer conservar el código de salida en *python*

La realización del trabajo práctico implicó una etapa previa de adquisición de conocimientos referidos a la construcción, lectura y normas para la elaboración de patrones de crochet. Consecuentemente, el primer gran paso en el desarrollo de Crochetify constó principalmente en aprender sobre esto, y poder abstraer sus normas generales a reglas de programación consistentes y que puedan ser luego traducidas a un lenguaje de programación con su debida estructura y componentes. La ayuda recibida por los docentes de la cátedra proveyeron material adicional y dieron explicaciones en vivo sobre cómo funciona el filet crochet, así como fue muy útil la colaboración de familiares aficionados de este mundo.

3.4. Dificultades Encontradas

A lo largo del desarrollo de este lenguaje de programación, se han encontrado distintas dificultades que obligaron a re-pensar muchos componentes de Crochetify. La principal dificultad encontrada se debió al manejo de estructuras y tipos abstractos de datos (ADTs) que fueron desarrollados en la etapa principal del proyecto, dónde se desconocía la totalidad de lo que provendría en el futuro. Dado que se decidió crear una estructura para todos los tipos diferentes de componentes que hacen a un patrón de crochet (como puntos, filas, colores, patrones, estructuras de control, etc.) pero que aún así todos estos componentes se debían guardar dentro de estructuras que las engloben (como una secuencia, por ejemplo), se presentaron ciertas complicaciones al tener que realizar averiguaciones de los tipos de dato y se debió lidiar con sus posteriores (y necesarios) casteos. Si bien éstos fueron la pieza fundamental que permiten acceder a la información y a los valores que previamente se habían almacenado en cada variable, el código se volvió rápidamente difícilmente legible y complicado de sostener a lo largo del desarrollo. Esta dificultad se debió al afán inicial de hacer todo el desarrollo lo más genérico posible, teniendo que sortear las dificultades que un lenguaje como C presenta si se desea trabajar de este modo.

Además, han surgido grandes momentos de aprendizaje (y también estrés) al momento de hacer las validaciones necesarias para la invocación de patrones, ya que dentro de Crochetify

existen cuatro maneras diferentes de hacer referencia a uno: de manera anónima (mediante el uso de corchetes), mediante una invocación sin parámetros y con parámetros, y hasta simplemente por su nombre identificador (por ejemplo, dentro de la declaración de otro patrón, si el primero fue recibido como argumento de este segundo). Esto implicó tener que lidiar con todos los escenarios posibles que pueden llegar a existir, teniendo que definir algunos límites, como por ejemplo dónde se podía hacer uso de patrones y dónde no. En línea con esto, el anidamiento de patrones también presentó desafíos que no se tuvieron en mente en un inicio, los cuales se supieron afrontar y superar con mucha paciencia y dedicación.

Finalmente, realizar el seguimiento de cada flujo y programa posible dentro de Crochetify implicó seguir bien de cerca la alocaión dinámica de la memoria necesaria y su posterior liberación, por lo que se han encontrado blockers al momento de continuar con el desarrollo del trabajo gracias a secuencias que presentaban algún problema en torno a esto. Para resolver estas trabas fue menester debuguear paso a paso lo que acontecía para lograr encontrar la causa del error y finalmente atacarla y resolverla.

3.5. Modificaciones respecto al diseño inicial

Al momento de diseñar las directivas, estructuras de control y funcionalidades del lenguaje para el primer diseño de Crochetify, fueron considerados ciertos aspectos que posteriormente se encontraron confusos y/o poco claros para los usuarios, y algunos otros los cuales se podían potenciar y explotar más. Es por ello que el lenguaje ha sufrido ciertas modificaciones con respecto a su versión original:

- MIRROR con patrones con nombre
 - Se eliminó la posibilidad de utilizar la estructura de control MIRROR junto a un patrón con nombre (una variable de tipo Pattern). Esto fue así ya que no se puede garantizar que una variable de este tipo solo contenga una única fila, por lo que la definición de lo que sería “espejar” un patrón de varias líneas resultaría ambigua y poco clara (no se sabría si esperarías en vertical, en horizontal, o a ambos lados ni si invierte el orden de las filas o simplemente se limita a los puntos de cada una). Para evitar esto, se permite hacer uso de la funcionalidad de MIRROR pero solo con patrones que sean anónimos, los cuales constan únicamente de una sola fila con sus puntos.
- Definición de patrones de una única línea
 - Para la elaboración del diseño inicial, estaba contemplada la posibilidad de definir variables de tipo Pattern mediante el uso de corchetes. Esto se volvía poco claro dado que esta misma notación era la empleada para crear aquellos patrones



denominados anónimos, y además resultaba innecesario y redundante ya que nada imposibilita definir patrones de una sola línea mediante llaves (como cualquier otro patrón), por lo que se decidió abarcar la definición de cualquier tipo de patrón mediante su escritura clásica.

- Colores predefinidos mediante nombres, con paréntesis
 - Inicialmente, Crochetify solo contaba con una cantidad reducida de colores aceptados para una fila, descritos mediante identificadores en inglés como (RED), (BLUE), (YELLOW), etc. Durante el desarrollo, se definió que esto limitaba el poder de expresión de los patrones de crochet a un conjunto de pocas tonalidades, cuando en realidad el mundo del tejido era tan amplio como colorido. Es por eso que se decidió aceptar colores en su formato hexadecimal para permitir a los diseñadores de crochet filet ingresar sus propios tonos y poder darle la apariencia que más se ajuste a sus tejidos, haciendo que el único límite sea su imaginación.

4. Futuras extensiones

Cómo se mencionó durante la introducción, Crochetify trata una técnica dentro del amplio mundo del crochet: el filet crochet. Consecuentemente, esto restringe enormemente no sólo los puntos utilizables si no también la utilidad del lenguaje, ya que el filet crochet se utiliza para construir patrones para, por ejemplo, manteles, cortinas, tapetes, entre otros.

A fin de sumarle utilidad al lenguaje, en un futuro podría extenderse de tal manera que se puedan realizar cada vez patrones de crochet más complejos, ampliando las posibilidades y permitiendo, por ejemplo, diseños tridimensionales o circulares. Para ello, sería necesario definir estructuras de control que permitan dibujos con distintos ángulos y no solo con líneas rectas, como así también ampliar la cantidad de puntos disponibles y, posteriormente, agregar los chequeos que sean necesarios para validar los patrones.

Otras cosas que se han considerado a lo largo del desarrollo del trabajo, y que se podrían incorporar como futuras extensiones de Crochetify involucran el agregado de valores por defecto para la definición de los patrones, exportación del patrón bajo otro formato y agregado de otras estructuras de control que puedan ser de interés dentro del mundo del crochet.



5. Conclusiones

El desarrollo de Crochetify permitió explorar el diseño e implementación de un lenguaje de programación específico (DSL) con un dominio delimitado. A lo largo del proceso, se presentaron desafíos relacionados con el diseño de estructuras genéricas, el manejo de memoria en C, la validación semántica de un lenguaje con componentes gráficos y la integración con herramientas externas para la generación de la salida visual.

El lenguaje resultante logra traducir de manera efectiva una descripción imperativa de un patrón de crochet filet en una representación gráfica que puede ser utilizada como guía real por personas aficionadas o profesionales del tejido. Crochetify cumple con los objetivos propuestos inicialmente, y sienta una base sólida para futuras extensiones que permitan ampliar tanto su potencia expresiva como su utilidad práctica.

6. Referencias

Attractive Chaos. (s.f.). **klib**: A generic library in C. GitHub. Recuperado en de Junio de 2025, de <https://github.com/attractivechaos/klib>

Joy of Motion Crochet. (s.f.). *Reading Crochet Charts*. Recuperado el 22 de junio de 2025, de <https://joyofmotioncrochet.com/reading-crochet-charts/>

7. Bibliografía

El siguiente material fue de utilidad para adquirir los conocimientos básicos sobre el mundo del crochet filet.

Crochet Parade. (s.f.). **Crochet Parade**. Recuperado en Junio de 2025, de <https://www.crochetparade.org/>

Crochetadas. (2014, mayo). **Filet para principiantes**. Blogspot. Recuperado de <https://crochetadas.blogspot.com/2014/05/filet-para-principiantes.html>

Street, K. (s.f.). **Freeform crochet grammar booklet (v2)** [Folleto]. Issuu. Recuperado en Junio de 2025, de https://issuu.com/kirastreet9/docs/freeform_crochet_grammar_booklet_v2